

CENTRALESUPÉLEC — UNIVERSITÉ PARIS-SACLAY

DATA-DRIVEN CONTROL CHALLENGE WEEK

INDUSTRIAL PARTNER: SAFRAN ELECTRONICS & DEFENSE

Data-Driven Control Report

Topic: Aerial add-on for a ground drone — modeling, identification and control.

Authors:

Victor ROUSSEL
Arthur ENRICO
Gianluca PAOLETTI

3–7 November 2025

Contents

1	Introduction and Project Description	1
1.1	Presentation of the Partner	1
1.2	Problem Statement and Expected Benefits	1
2	Data and Tools	3
2.1	Provided Dataset and Signals	3
2.2	Software Stack	3
3	Methodologies and Analysis	4
3.1	Raw Data Inspection	4
3.2	Data Validation and Outlier Screening	4
3.3	Sampling Period Justification	5
3.4	Modeling Approach	6
3.5	Validation of the Neural Model	8
4	Model Predictive Control and State Estimation	10
4.1	Motivation and Objectives	10
4.2	Model Predictive Control	10
4.3	Results of the MPC Applied to the Neural Model	11
4.4	State Estimation with an Extended Kalman Filter	12
4.5	Integrated Simulation and Discussion	13
5	Conclusion	14

List of Figures

1.1	Illustration of the three-step UAV mission: take-off, observation and landing.	2
3.1	Raw output signals. The visible spikes correspond to abnormal measurements.	4
3.2	Minimum output value across channels as a function of time. Four points are identified as outliers.	5
3.3	Sample of iteration 4, showing a transition time of about 0.5 s.	6
3.4	Least-squares fit on a narrow operating range: measured output (blue) against fitted model (red).	6
3.5	Measured and predicted outputs over a wider operating range.	7
3.6	Relative error between normalised measured and predicted accelerations. The maximum is about 10 %.	7
3.7	Measured and predicted accelerations for the LSTM model.	8
3.8	Conceptual comparison between the RNN (left) and LSTM (right) architectures.	8
3.9	Response of the neural model and of the system to the same reference.	9
4.1	Simulink implementation of the MPC control architecture.	10
4.2	Closed-loop response of the neural model under MPC.	11
4.3	Simulink implementation of the extended Kalman filter, integrated with the MPC and the UAV model.	12
4.4	Reference z_{ref} and system output z for the complete control loop.	13

Chapter 1

Introduction and Project Description

1.1 Presentation of the Partner

The industrial partner for this project is **Safran Electronics & Defense**, a French company specialised in aerospace and defence technologies. Safran develops high-precision systems for navigation, guidance and autonomous vehicles. Within the framework of the *Data-Driven Control Challenge Week* at CentraleSupélec, the company proposed a case study related to autonomous aerial and ground systems.

The academic supervision was provided by researchers from CentraleSupélec, Université Paris-Saclay, in collaboration with Safran's teams.

1.2 Problem Statement and Expected Benefits

As the development of the ground drone reaches completion, a new enhancement is envisioned: extending its perception capabilities through an aerial viewpoint. Safran engineers highlighted a key limitation of the current system: ground drones have a limited field of view for obstacle detection, and an aerial perspective would improve this capability.

The proposed solution consists in adding a **quadrotor drone** on a platform attached to the back of the ground vehicle. The drone must be able to **take off**, **gather data from above**, and then **land back on the platform**. A schematic representation of the mission sequence is shown in Figure 1.1.

However, the UAV was provided with **no technical documentation**. The team therefore had to rely solely on a **data-driven control approach** to model and control the system dynamics. The available dataset contains input and output measurements, but the sensors used are known to be unreliable. A preliminary step was therefore to perform **data validation** and **outlier removal** before proceeding with identification and control design.

The work carried out during this challenge includes:

1. **Model definition:** justify the chosen sampling period ($T_s = 0.05\text{ s}$) and select an appropriate model structure.
2. **Data validation:** handle missing or inconsistent values, detect and remove outliers, normalise the signals.
3. **System identification:** fit and evaluate a model using the provided datasets.
4. **Control design:** develop and tune a control law enabling the UAV to perform the three-step mission (take-off, observation, landing).

The expected outcome is a **data-driven model** of the UAV, together with a **control strategy** enabling safe and precise operation.

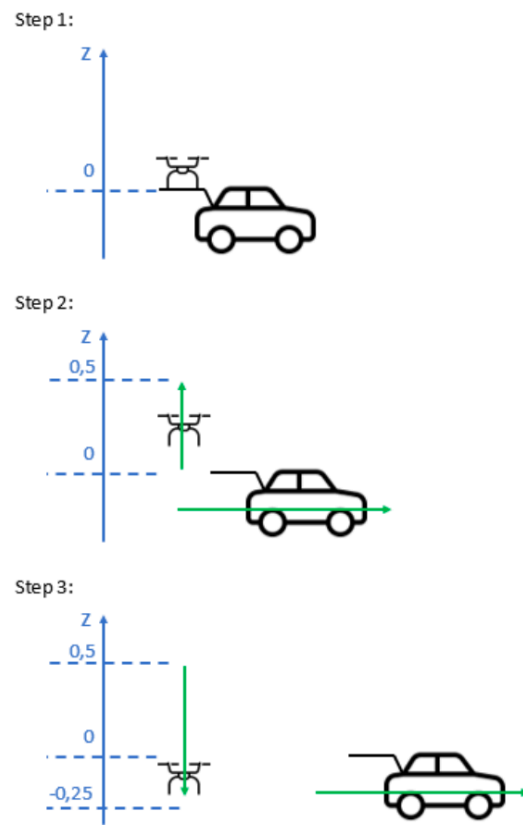


Figure 1.1: Illustration of the three-step UAV mission: take-off, observation and landing.

Chapter 2

Data and Tools

2.1 Provided Dataset and Signals

The project relies entirely on data-driven modeling and control; the available datasets therefore constitute the foundation of the study. Three files were provided at the beginning of the challenge:

- `bdd_in.csv`, containing the **input signals** applied to the UAV actuators.
- `bdd_out.csv`, containing the **output signals**, i.e. the measured dynamic response of the system.
- `UAV_model.slx`, a Simulink model of the UAV system used for simulation and controller integration.

The output file contains several measured channels, among which the vertical acceleration a_z , the attitude and the position estimates. The linear model identified in Chapter 2 reproduces all of them, whereas the neural model retained for control predicts the vertical acceleration alone. The vertical position z , which is the quantity finally controlled in Chapter 3, is then obtained from a_z by two successive integrations.

Before any modeling step, the datasets were imported and merged into synchronised time-series structures. Preliminary inspection revealed a small number of **outliers** and occasional sensor noise peaks, confirming the need for a dedicated data validation stage.

2.2 Software Stack

The project required a combination of data analysis, machine learning and control design tools. The following software environment was used:

- **Python**, for data preprocessing, visualisation and neural network modeling. The main libraries used were NumPy, pandas, Matplotlib and PyTorch.
- **MATLAB / Simulink**, for system simulation and control validation. The neural network weights obtained in Python were exported and integrated into Simulink for testing.
- **Git and Google Colab / Google Drive**, for version control and team collaboration.
- **L^AT_EX (Overleaf)**, for the preparation of the presentation and of this report.

The data-driven workflow combined both environments:

1. Python was used for **dataset inspection, cleaning and model identification**.
2. MATLAB/Simulink was used for **controller implementation and validation on the UAV model**.

Chapter 3

Methodologies and Analysis

3.1 Raw Data Inspection

Before starting the modeling process, the first step consisted in inspecting the raw datasets provided by the client. The goal was to assess their overall quality and to detect possible inconsistencies, missing values or measurement artefacts.

The dataset is composed of two synchronised time series:

- **Inputs** (u): the control signals applied to the UAV motors.
- **Outputs** (y): the measured vertical acceleration a_z of the drone.

All output channels were plotted together in order to visualise the global behaviour of the system. Most signals lie within consistent dynamic ranges, while a few channels exhibit sharp peaks and discontinuities. These anomalies are characteristic of sensor noise or acquisition faults and must be treated before any identification step.

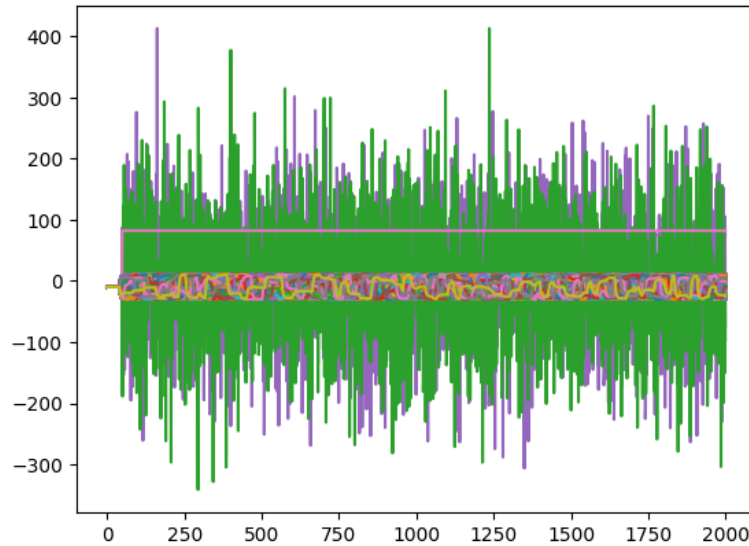


Figure 3.1: Raw output signals. The visible spikes correspond to abnormal measurements.

Figure 3.1 confirms that an outlier screening step was required prior to model identification.

3.2 Data Validation and Outlier Screening

Data validation aimed at removing unreliable samples and at preparing a consistent dataset. The criterion retained is the **minimum value over all output channels at each time step**:

$$m_t = \min_i y_i(t).$$

Abnormal negative peaks of m_t revealed four erroneous timestamps, corresponding to values incompatible with the physical limits of the system. The samples such that

$$m_t \in (-50, -20)$$

(in the units of the raw measurements) were flagged as invalid and removed from both the input and the output datasets:

$$\mathcal{D}_{\text{clean}} = \mathcal{D}_{\text{raw}} \setminus \{ t_k : m_{t_k} \in (-50, -20) \}.$$

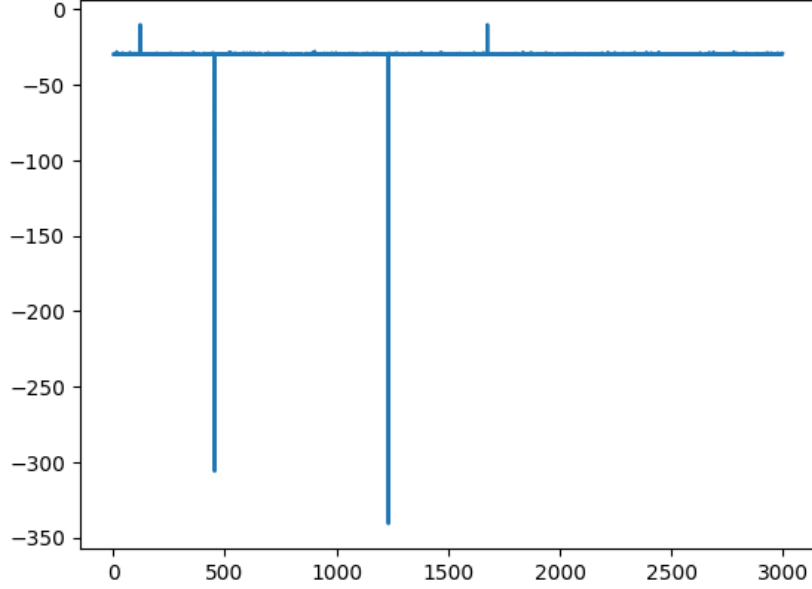


Figure 3.2: Minimum output value across channels as a function of time. Four points are identified as outliers.

After this step the datasets were realigned and normalised, which produced a consistent base for identification. This step ensures that the residuals and estimation errors reported later reflect model imperfections rather than data corruption.

3.3 Sampling Period Justification

From the measured system response, the UAV reaches its steady state after approximately 0.5 s. The usual design rule requires a sampling period at least ten times smaller than the dominant time constant:

$$T_s \leq \frac{T_{\text{system}}}{10} = \frac{0.5 \text{ s}}{10} = 0.05 \text{ s}.$$

The upper bound of this interval was retained, since all the models trained during the challenge remained accurate at this rate:

$$T_s = 0.05 \text{ s}.$$

This choice is a compromise between numerical precision, computational load and controller responsiveness during simulation.

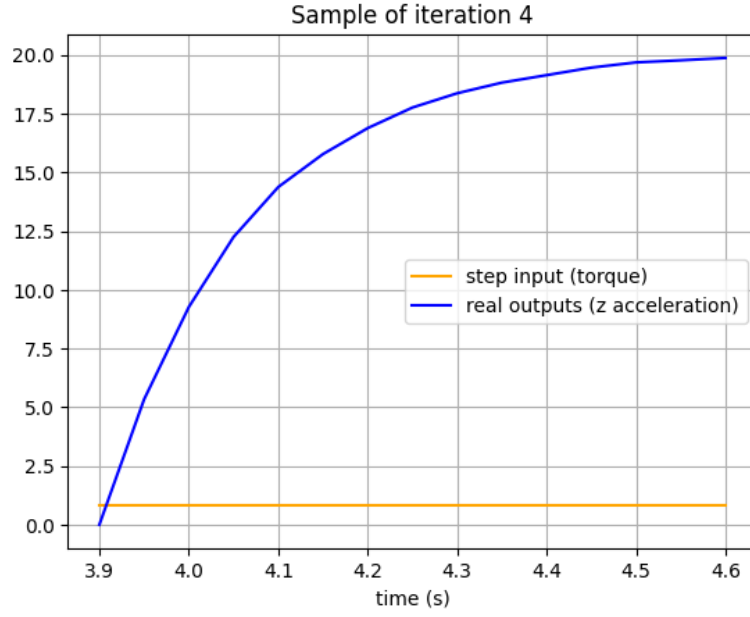


Figure 3.3: Sample of iteration 4, showing a transition time of about 0.5 s.

3.4 Modeling Approach

Three modeling strategies were evaluated and compared, all identified with the same sampling period $T_s = 0.05$ s:

1. A **first-order linear model**, used as a fast and interpretable baseline.
2. A **long short-term memory network (LSTM)**, aimed at capturing higher-order and nonlinear effects.
3. A **simple recurrent neural network (RNN)**, retained for the control stage.

First-Order Linear Model

The first modeling step consisted in fitting a **first-order linear approximation** of the UAV dynamics by least squares. This model is easy to identify from data and provides a baseline for quick simulations and for controller design.

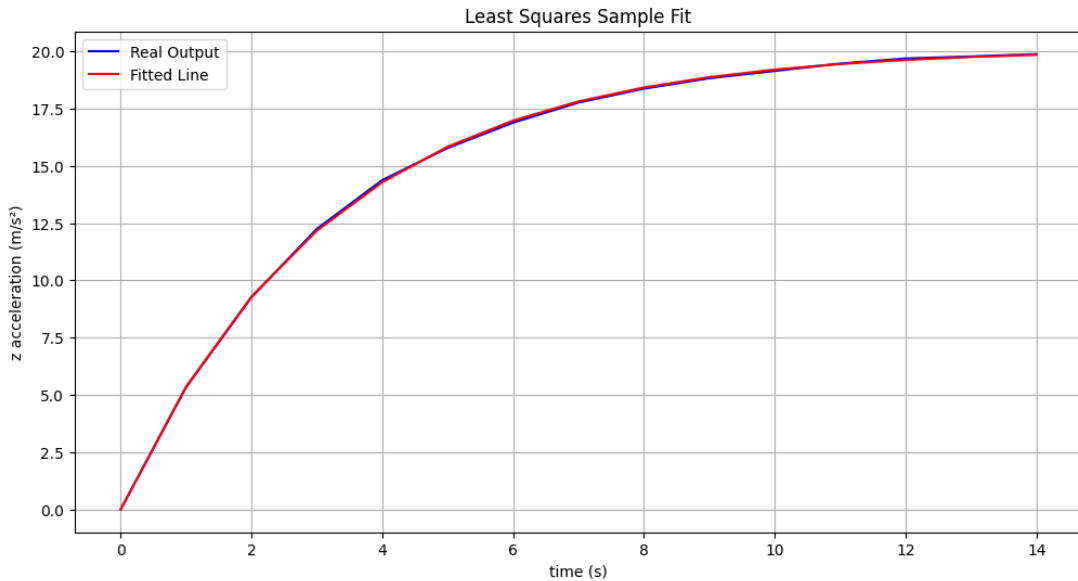


Figure 3.4: Least-squares fit on a narrow operating range: measured output (blue) against fitted model (red).

After verifying its consistency on a limited dataset, the identification was extended to a wider range of operating conditions (Figure 3.5). The relative error then reaches about 10 % at the peaks (Figure 3.6), which motivated the use of a nonlinear model.

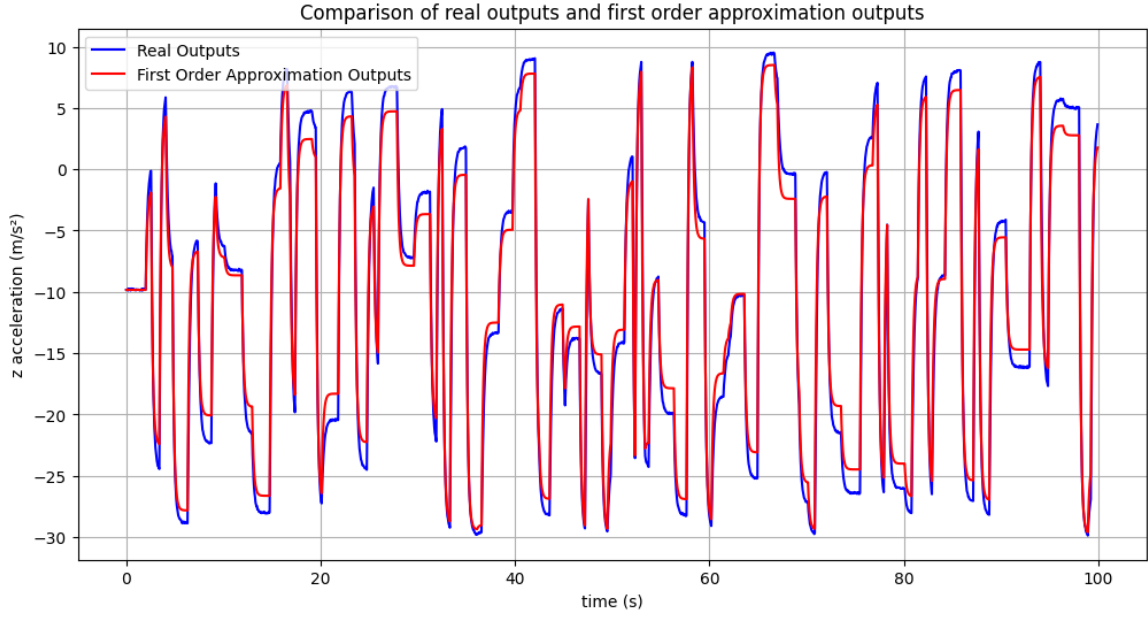


Figure 3.5: Measured and predicted outputs over a wider operating range.

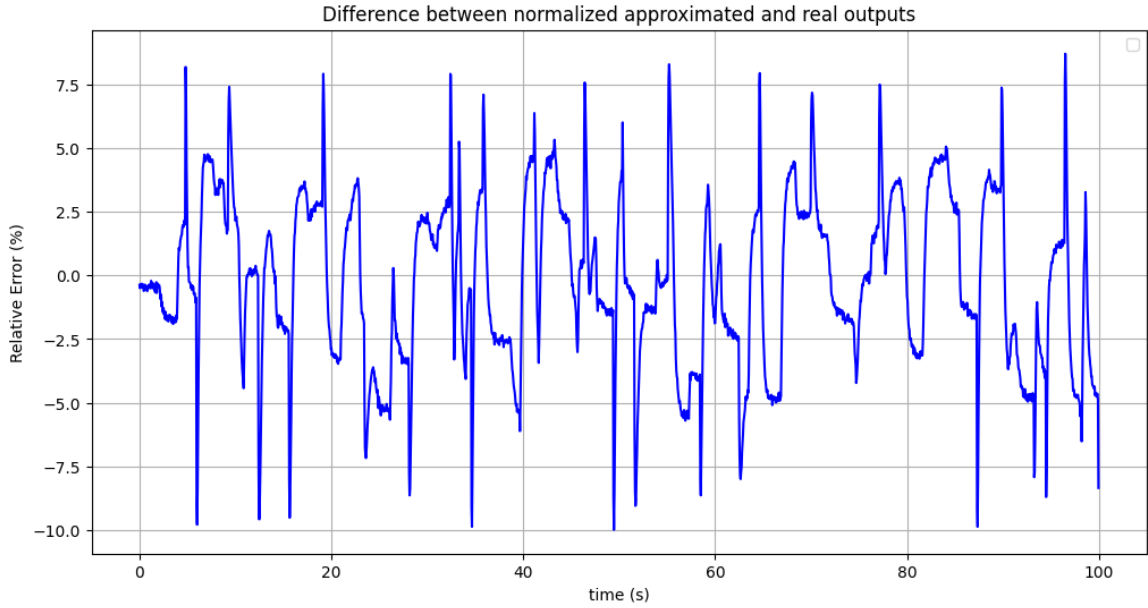


Figure 3.6: Relative error between normalised measured and predicted accelerations. The maximum is about 10 %.

Nonlinear Model: LSTM Network

The linear model captures the average trend but not the fast transients, which motivated a nonlinear model. A **long short-term memory network (LSTM)** was trained on the cleaned dataset to predict the vertical acceleration a_z from the input sequence. The recurrent structure is suited to this task since the response of the UAV at a given instant depends on the recent history of the commands and not only on their current value.

The absence of overfitting was monitored during training by following the loss on a held-out validation split alongside the training loss. The two curves remained of the same order throughout, which indicates that the network generalises to unseen sequences rather than memorising the training set.

Figure 3.7 compares the measured and predicted accelerations. The prediction follows the measurement closely, including during the transients where the linear model degrades.

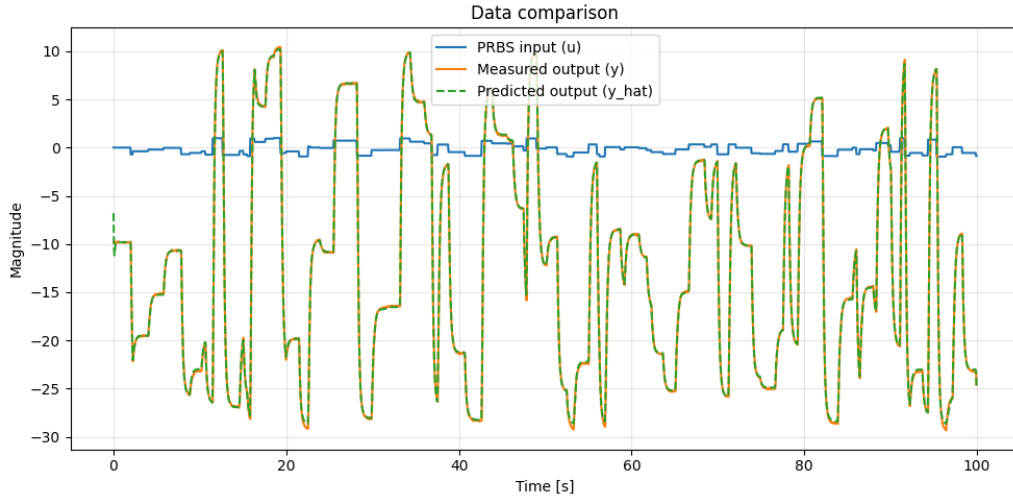


Figure 3.7: Measured and predicted accelerations for the LSTM model.

Transition to the RNN Model

A simple **recurrent neural network (RNN)** was then trained under the same conditions, in order to test whether the gating mechanism of the LSTM was necessary on this problem. The two architectures differ by the internal structure of the recurrent cell, as illustrated in Figure 3.8: the LSTM maintains a separate cell state controlled by three gates, whereas the RNN propagates a single hidden state through one nonlinearity.

The prediction performance of the two models turned out to be comparable on the test sequences. The RNN being significantly lighter, both in number of parameters and in inference cost, it was retained for the control stage. This choice also simplifies the integration in Simulink, since the prediction model is evaluated at every sampling step inside the controller.

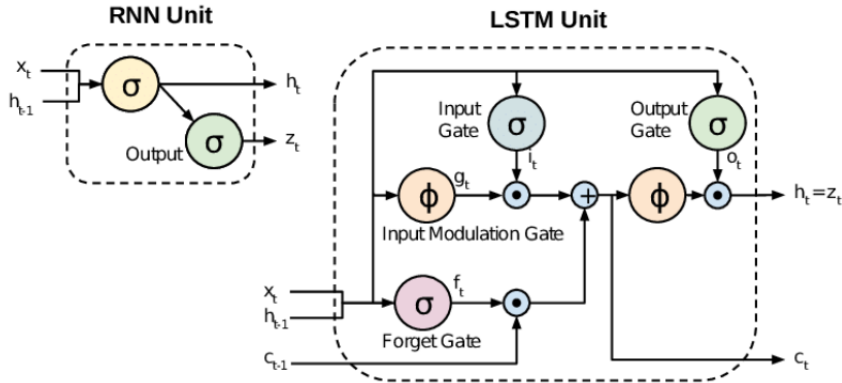


Figure 3.8: Conceptual comparison between the RNN (left) and LSTM (right) architectures.

3.5 Validation of the Neural Model

The retained model must now be validated, in order to check that it generalises and that it can be used inside the control loop.

Generalisation was checked during training itself: the loss computed on a held-out set was monitored at each epoch alongside the training loss. Both remained of the same order and decreased together, without the divergence between the two curves that characterises overfitting. The model therefore reproduces the dynamics of the system rather than the particular sequences it was trained on.

The model must also remain accurate on input signals simpler than those of the training set, which are the ones it will receive once placed in the loop.

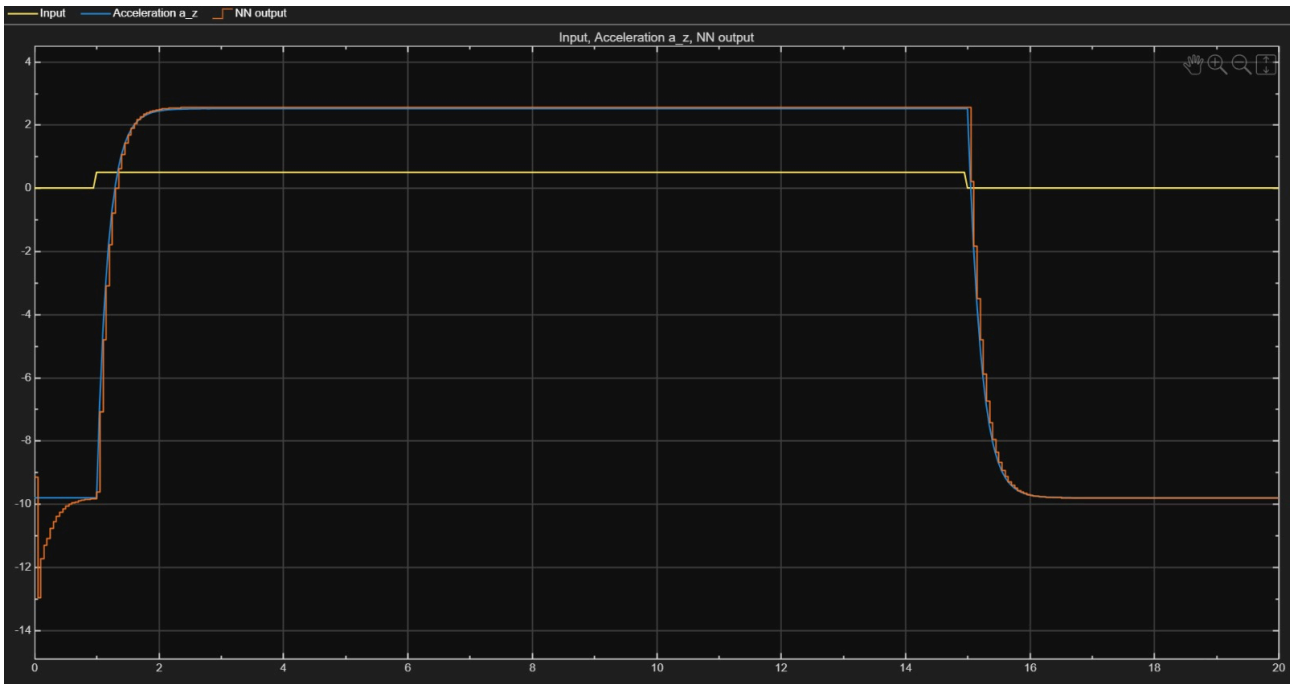


Figure 3.9: Response of the neural model and of the system to the same reference.

Figure 3.9 shows the output a_z of both the system and the neural model for the same input. The model follows the measured curve closely, which validates its use in the control loop.

Chapter 4

Model Predictive Control and State Estimation

4.1 Motivation and Objectives

After validating the linear and the neural models, the next phase of the project focused on designing a control architecture ensuring stable and precise UAV operation. Given the measurement noise, the actuator limits and the nonlinear dynamics, a **nonlinear Model Predictive Control (MPC)** strategy was adopted, using the identified RNN as its internal prediction model. This control law computes the optimal control actions over a finite horizon while respecting the constraints on the state and on the input.

The quantity to be controlled is the altitude z , whereas the neural model predicts the vertical acceleration a_z . The two are related by two successive integrations, $a_z \rightarrow v_z \rightarrow z$, performed inside the control loop.

The internal state of the RNN is not measured, and the MPC needs it to initialise its predictions at every sampling step. An **extended Kalman filter (EKF)** was therefore integrated into the loop to estimate it from the available measurements, and at the same time to reject sensor noise.

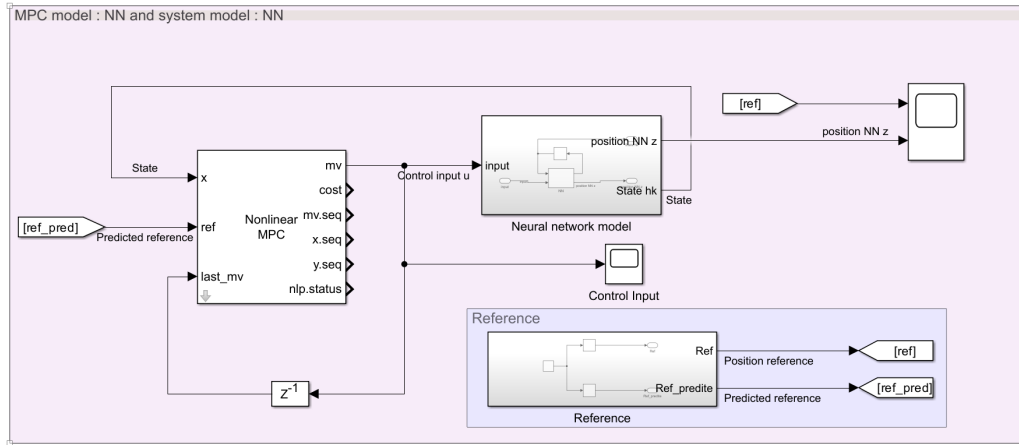


Figure 4.1: Simulink implementation of the MPC control architecture.

4.2 Model Predictive Control

Principle

MPC determines the control sequence by minimising, at each sampling instant k , a cost function over a finite horizon. Since the objective is to track an altitude reference, the cost penalises the tracking error on the output and the variations of the control input:

$$J_k = \sum_{i=1}^{N_p} (y_{k+i} - r_{k+i})^\top Q (y_{k+i} - r_{k+i}) + \sum_{i=0}^{N_c-1} \Delta u_{k+i}^\top R \Delta u_{k+i}, \quad \Delta u_k = u_k - u_{k-1},$$

subject to the model dynamics and to the constraints on the state and on the control. Penalising the increment Δu rather than u itself allows a non-zero command in steady state, which is required here since the UAV must compensate its own weight in order to hold an altitude.

Only the first control action u_k of the optimal sequence is applied to the system, and the optimisation is repeated at the next sampling step (*receding horizon* strategy). The predictions y_{k+i} are obtained by propagating the RNN over the horizon, which makes the optimisation problem nonlinear and requires it to be solved numerically at each step.

Implementation in Simulink

The main parameters retained are:

- Sampling period: $T_s = 0.05$ s
- Prediction horizon: $N_p = 15$
- Control horizon: $N_c = 15$
- Weights, scalar since a single output is controlled: $Q = 3$ and $R = 1$

Physical limits on the control inputs were imposed to guarantee feasibility. This setup is a compromise between tracking precision and actuation smoothness.

4.3 Results of the MPC Applied to the Neural Model

The tuning was obtained by successive trials; the resulting closed-loop response is shown in Figure 4.2.

The MPC controller based on the RNN model tracks the reference correctly. At the beginning of the experiment the drone briefly moves downwards before converging towards the reference. This initial deviation may originate from a bias in the training data, where all sequences start at zero, or from the fact that the drone is initialised as if released in free fall. It is amplified by the double integration, since a constant error on the predicted acceleration produces a quadratic drift on the reconstructed altitude.

When the reference rises to $z = 5$ m, the response is fast and accurate, with a small overshoot before settling. The closed-loop dynamics therefore remain slightly under-damped, but the system stabilises quickly and holds the target altitude with a small steady-state error.

Improvements could focus on reducing the initial drop and the overshoot, through a better initialisation, richer training data, or a retuning of the MPC weights.

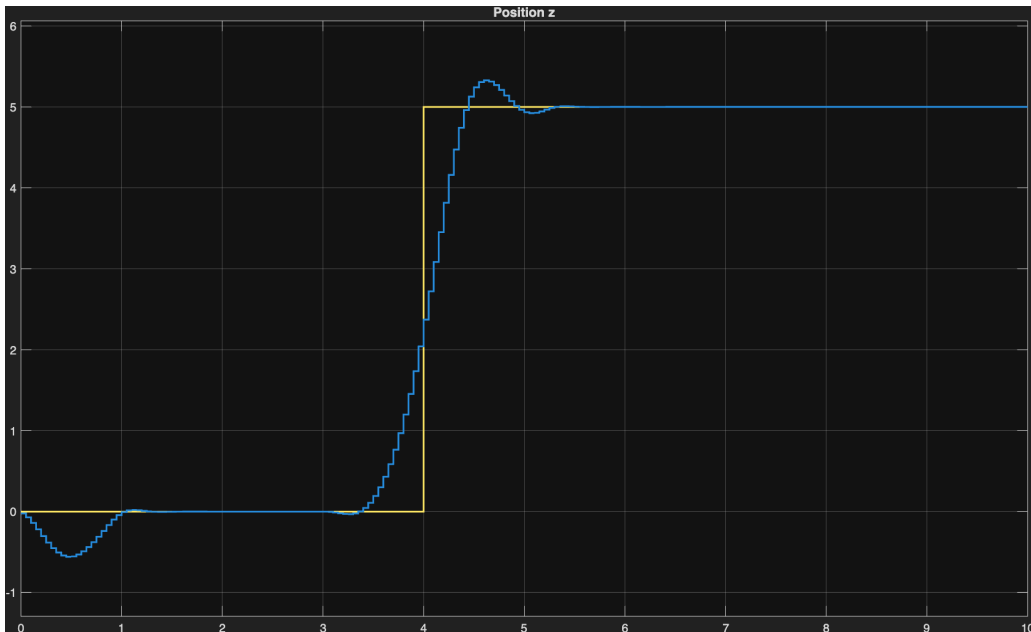


Figure 4.2: Closed-loop response of the neural model under MPC.

4.4 State Estimation with an Extended Kalman Filter

Principle

The prediction model being a recurrent network, its internal state is a hidden vector that is not directly observed. An **extended Kalman filter** was used to estimate it, since the standard Kalman filter applies only to linear dynamics. The system is written in the general nonlinear form

$$x_k = f(x_{k-1}, u_{k-1}) + w_k, \quad z_k = h(x_k) + v_k,$$

where x_k denotes the hidden state of the RNN, f its recurrence relation and h its output layer, which produces the predicted vertical acceleration. The terms w_k and v_k are zero-mean Gaussian process and measurement noises, with covariance matrices Q_w and R_v . These matrices are distinct from the MPC weights Q and R introduced above.

The filter alternates a prediction step and a correction step, the covariance being propagated through the Jacobians of f and h evaluated at the current estimate:

$$F_k = \left. \frac{\partial f}{\partial x} \right|_{\hat{x}_{k-1|k-1}, u_{k-1}}, \quad H_k = \left. \frac{\partial h}{\partial x} \right|_{\hat{x}_{k|k-1}}.$$

Prediction: $\hat{x}_{k|k-1} = f(\hat{x}_{k-1|k-1}, u_{k-1})$
 $P_{k|k-1} = F_k P_{k-1|k-1} F_k^\top + Q_w$

Correction:
$$\begin{aligned} K_k &= P_{k|k-1} H_k^\top (H_k P_{k|k-1} H_k^\top + R_v)^{-1} \\ \hat{\mathbf{x}}_{k|k} &= \hat{\mathbf{x}}_{k|k-1} + K_k (z_k - h(\hat{\mathbf{x}}_{k|k-1})) \\ P_{k|k} &= (I - K_k H_k) P_{k|k-1} \end{aligned}$$

The covariances Q_w and R_v were tuned empirically, by adjusting the trade-off between noise rejection and convergence speed of the estimate.

Simulink Integration

The filter was implemented within the UAV Simulink environment, as shown in Figure 4.3. It receives the measured output and the control input, and performs the prediction and correction steps at each iteration. The estimated state is then passed to the MPC block, which uses it to initialise its predictions over the horizon.

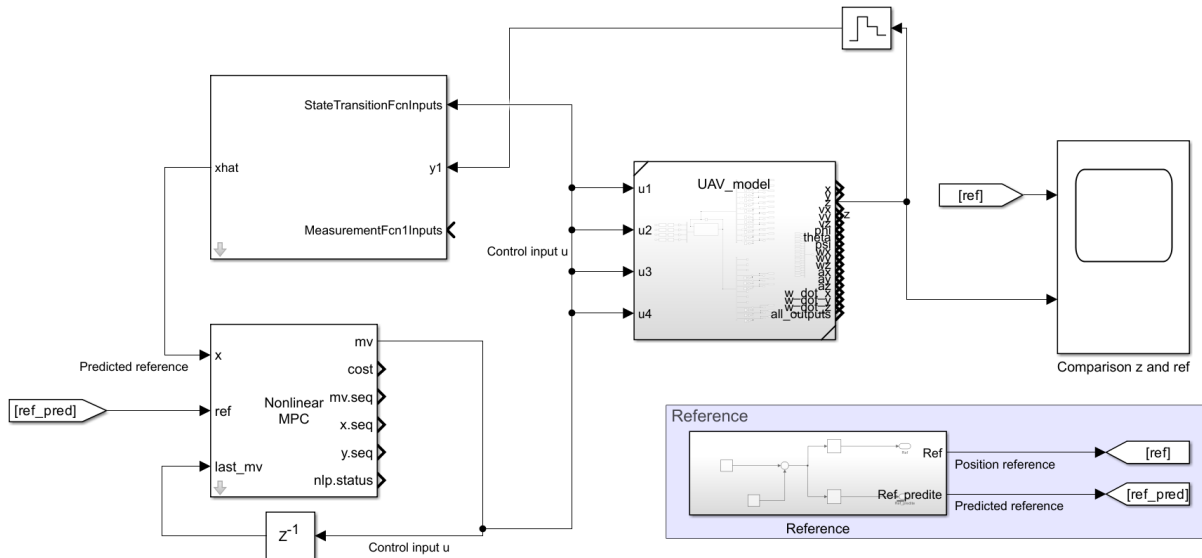


Figure 4.3: Simulink implementation of the extended Kalman filter, integrated with the MPC and the UAV model.

Simulation results show that the filter removes most of the measurement noise while keeping a fast convergence and a limited delay on the estimated state.

4.5 Integrated Simulation and Discussion

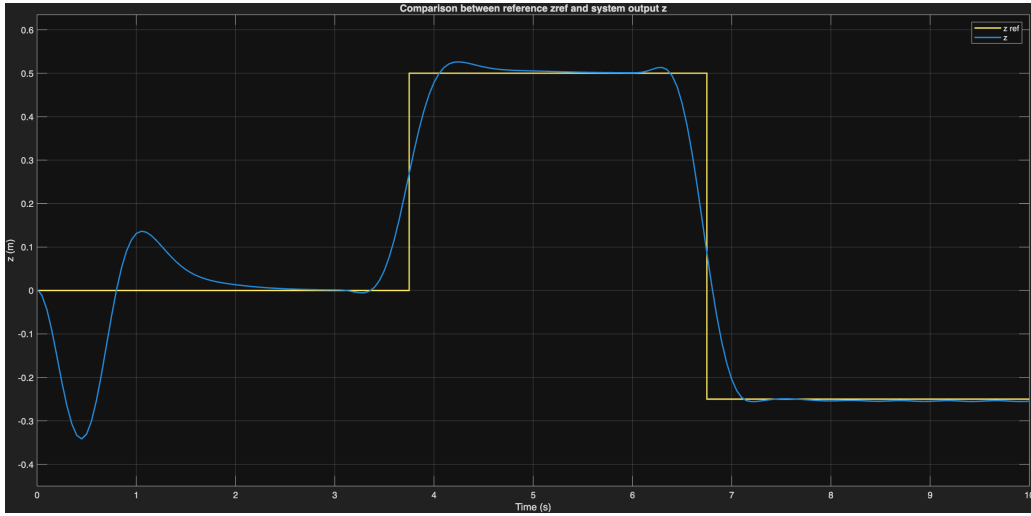


Figure 4.4: Reference z_{ref} and system output z for the complete control loop.

The final UAV control loop combines the nonlinear MPC and the extended Kalman filter. In Figure 4.4, the output z follows the reference with good accuracy: each reference step is tracked after a short transient, which shows that the MPC computes feasible control actions within the imposed input constraints.

The response remains smooth, which indicates that the filter provides a clean state estimate to the controller. The closed-loop system is stable and well damped, with bounded control effort.

Chapter 5

Conclusion

This project shows the feasibility of controlling a UAV through a fully data-driven workflow. Starting from raw input and output measurements, we carried out data validation, signal cleaning, sampling-period justification, model selection and neural network identification, before designing and tuning a model predictive controller integrated with a Kalman filter.

A first-order linear model provided an interpretable baseline and enabled a first exploration of the system dynamics. Its limited accuracy, with a relative error reaching about 10 %, motivated the development of recurrent models, which captured the nonlinear effects better and gave a lower prediction error on unseen data. The LSTM and the RNN reached comparable performance on the test sequences; the RNN was retained for the control stage, being significantly lighter and simpler to integrate in Simulink.

The nonlinear MPC designed on top of the RNN model tracked the altitude reference with good stability and a small steady-state error. Its integration with an extended Kalman filter, estimating the hidden state of the network, improved robustness against measurement noise. Some transient imperfections remain, in particular an initial downward motion and a small overshoot, which we attribute to a bias in the training data and to an incomplete state initialisation.

Future Improvements

- **Improved numerical integration:** the altitude being reconstructed from the predicted acceleration by two integrations, a more accurate integration scheme would limit the accumulation of the prediction error, in particular during fast transients.
- **Retraining on shifted datasets:** retraining the network on data cropped at the beginning of each sequence, where the control input is zero, would mitigate the bias responsible for the early tracking deviation.
- **Extended state with integral action:** adding an integral action would reduce the steady-state error and increase robustness to model mismatch.

This work combines machine-learning models with a predictive control strategy. Despite the absence of physical specifications and the presence of imperfect measurements, the proposed approach achieved closed-loop control of the UAV over the three-step mission.